# EurekaVault Comprehensive Implementation Reference

Implementation-derived documentation

Repository snapshot: 18 August 2026

## Contents

# 1 Document Basis and Scope

This reference documents the EurekaVault implementation inspected at Git commit `6818b808dc51cf72d3698e61a6bfbfa059108f46`. It is intentionally implementation-led. Current TypeScript code, Firebase rules, Netlify Functions and Git history are treated as authoritative for current functionality. Older repository prose is retained as historical evidence only when it disagrees with the implemented system.

The goal of this document is completeness rather than presentation polish. It records what the system does, how the major pieces interact, what data is persisted, what is sent to AI services, the important limits/invariants, and when major functionality entered the repository.

## 1.1 Current product characterization

EurekaVault is a private Prompt knowledge and evolution workspace with hierarchical organization, automatic Prompt-local version history, Global Version baselines, attachments, Prompt lineage, Mindsets, Preferences, Decisions, engagement tracking, deterministic search, and a set of authenticated Gemini-powered retrieval/transformation/synthesis tools. The newest AI-side capability in the inspected repository is the user-confirmed Semantic Prompt Finder feedback loop added on 18 August 2026.

## 1.2 Important documentation drift

The root README and in-application Roadmap still contain early Release 1 statements saying that AI calls are not implemented. Those statements are no longer true. The root README also mentions unlimited nested folders, while the current domain model contains no Folder entity. This reference documents the current direct hierarchy: Endeavor → Task → Prompt → Prompt Version.

# 2 Technology Stack

- React 19 with TypeScript.
- Vite 8 for local development and production bundling.
- Firebase Authentication for Email/Password identity.
- Firebase Realtime Database for user-scoped persistence.
- React Router for browser routes.
- Motion for reduced-motion-aware animation/layout transitions.
- Lucide icons and Sonner notifications in the UI.
- Vitest for unit tests and ESLint for linting.
- Netlify for frontend hosting, SPA routing and server-side functions.
- Gemini Interactions API for the implemented AI features.

The package requires Node 22.12 or newer; Netlify is explicitly configured to use Node 22.12.0.

# 3 Runtime Architecture

The browser is the main application runtime. Authentication is isolated in `src/context/AuthContext.tsx`. Vault subscriptions and most data mutations are centralized in `src/context/VaultContext.tsx`. Shared record UI operations are coordinated by `EntityUiProvider`. The authenticated page tree is wrapped by `AppShell`.

## 3.1 Three runtime trust boundaries

1. **Browser**: React UI, deterministic search, local diffing, payload construction and Firebase SDK access.

2. **Firebase**: authentication and owner-scoped Realtime Database persistence.
3. **Netlify Functions**: server-side verification of Firebase sessions and secret-bearing Gemini calls.

### 3.2 Authenticated routes

| Route | Purpose |
|---|---|
| `/dashboard` | Workspace overview. |
| `/achievements` | Activity-derived achievement progress/unlocks. |
| `/ai/find-prompt` | Gemini Semantic Prompt Finder and feedback learning. |
| `/ai/prompt-mixer` | Multi-Prompt synthesis. |
| `/ai/repurpose-prompt` | Controlled Prompt transformation. |
| `/hierarchy` | Endeavor/Task/Prompt hierarchy view. |
| `/prompts` | Prompt library. |
| `/prompts/:promptId` | Prompt Workspace. |
| `/relationships` | Prompt lineage graph and ledger. |
| `/mindsets` | Mindset CRUD/scoped records. |
| `/mindset-construction` | Deterministic Mindset Builder. |
| `/preferences` | Scoped Preferences. |
| `/versions` | Global Versions. |
| `/commits` | Legacy alias redirecting to Global Versions. |
| `/decisions` | Open/finalized Decision records. |
| `/roadmap` | In-app roadmap page (contains stale AI gating text at this snapshot). |
| `/archive` | Archived record management. |
| `/settings` | Workspace identity, diagnostics, verification and JSON export. |
| `/search` | Deterministic Prompt/history word search. |

## 4 Authentication and Workspace Ownership

Firebase Email/Password authentication supports sign-up, sign-in, sign-out, password reset, verification email and verification resend. New account creation also initializes the workspace profile and seeded Decisions under the new Firebase UID.

The authoritative EurekaVault path is `intellectVault/users/{{uid}}`. Database rules require the authenticated UID to equal the path UID for reads and writes. The current architecture is therefore a private owner workspace, not a shared collaborative vault.

The public Firebase web configuration is project identification rather than authorization. Authorization depends on the Firebase session and server-enforced Realtime Database rules.

## 5 Persisted Data Model

Most content records contain an ID, created/updated timestamps, created/updated user stamps and optional archive metadata. The current user root can contain profile, activity days/stats, achievement unlocks, Endeavors, Tasks, Prompts, Prompt Versions, attachments, relations, Finder feedback, Mindsets, Preferences, legacy local commits, Global Versions and Decisions.

### 5.1 Workspace Profile

Stores workspace name, owner name, owner email and timestamps. Settings can update the workspace name.

### 5.2 Endeavor

Top-level area of work. Stores name, description and a manual agentic summary.

### 5.3 Task

Belongs to one Endeavor. Stores name, description, purpose, Endeavor ID and a manual suggested-improvement field.

### 5.4 Prompt

Primary artifact. Stores title, description, purpose, full content, Task ID and four manually editable analysis/context fields: agentic summary, suggested improvement, AI evaluation and generated context. These stored manual fields do not automatically call an AI model.

### 5.5 Prompt Snapshot and Prompt Version

A snapshot preserves title, description, purpose, content, Task assignment and the manual analysis/context fields. Prompt Version records carry Prompt ID, version label/number, content/snapshot, change description, changed fields and source/change-type metadata. Legacy local-commit linkage remains for compatibility with earlier design.

### 5.6 Prompt Attachment

Stores Prompt ID, filename, MIME type, byte size and Base64 payload. Current limits are 2 MiB/file, 20 files/Prompt and 10 MiB total/Prompt.

### 5.7 Prompt Relation

Stores parent Prompt ID, child Prompt ID and the fixed relationship type `inspired-by`. Parent means the inspiring Prompt; child means the Prompt that was inspired.

### 5.8 Prompt Finder Feedback

Stores historical Finder query, confirmed selected Prompt ID, a title snapshot, returned match IDs/scores, model, corpus size and number of learning examples used. This record becomes potential evidence for future retrievals.

### 5.9 Mindset

Stores title, content, scope type/scope ID, a manual AI-generated-mindset field, optional source Prompt IDs and optional construction method. Current scope types include global, Endeavor, Task and Prompt.

### 5.10 Preference

Stores title, instruction, scope type and scope ID. Current Preference scope type is global, Endeavor or Task; Prompt-level Preference scope is not defined.

### 5.11 Decision

Stores title, category, Open/Finalized status, question, resolution and notes. Provider normalization upgrades several early seeded decisions to later finalized implementation semantics.

### 5.12 Global Version

The persisted type is still named GlobalCommit, but the current UI calls these Global Versions. A Global Version stores a display ID, title, author, summary, timestamp, optional version number, record counts and a complete snapshot.

## 6 Hierarchy and Organization

The current content hierarchy is direct:

$$\text{Endeavor} \rightarrow \text{Task} \rightarrow \text{Prompt} \rightarrow \text{Prompt Version}$$

No Folder type exists in the current TypeScript domain model. Prompt relationships are intentionally separate from hierarchy membership, allowing a Prompt in one Endeavor to inspire a Prompt in another Endeavor without changing either Prompt's Task/Endeavor assignment.

## 7 CRUD, Archive, Restore and Permanent Delete

The application supports normal create/read/update flows for implemented content collections and uses archive metadata for recoverability. Archived records can be restored where supported. Permanent deletion is implemented but guarded by dependency checks so referenced parent records are not removed while dependent records still exist. Prompt-specific lifecycle behavior also has to account for Prompt Versions, attachments and relationships.

The current behavior supersedes the earliest documentation that treated permanent historical deletion as an unresolved topic.

## 8 Prompt Library and Prompt Workspace

The Prompt Workspace is the primary artifact editing environment.

### 8.1 Editor tab

The editor maintains a draft containing Prompt metadata/content. A serialized saved snapshot is used for dirty-state detection. The UI displays Saving, Unsaved or Saved state. Browser unload is intercepted when unsaved work exists. Ctrl/Cmd+S triggers save.

### 8.2 History tab

Prompt Versions are sorted newest-first. The user chooses two versions to compare. Historical restoration takes the selected snapshot and sends it through the normal Prompt update path, thereby producing another new current version.

### 8.3 Files tab

Shows active Prompt attachments, used byte/count totals and upload controls. Files can be dragged/selected, validated, persisted, downloaded and removed.

### 8.4 Relationships tab

Shows incoming *inspired by* and outgoing *inspires* lineage. A relationship can be created, edited or removed. Candidate filtering and library-level validation reject invalid/cyclic relationships.

### 8.5 Toolbar and artifact actions

The Prompt Workspace supports focus mode, detail inspector toggle, copy Prompt content, AI Repurpose, AI Mix, duplicate, archive, delete and save.

## 9 Prompt-Local Version Control

Prompt history became an explicit product capability on 7 August 2026 in commit `417b29d` (*Adding copy, versioning*). The current workflow is automatic: Prompt creation establishes Version 1 and meaningful saved changes create later snapshots.

Version restoration is append-only in effect: restoring old content creates a new version rather than deleting the intervening history.

### 9.1 Git-style line diff

Commit `1ac483e` on 8 August 2026 added the line-diff feature. The diff library provides additions/removals, exact old/new line numbers, unified and side-by-side modes, compact unchanged context and a fallback strategy for very large comparisons. This is Git-inspired review behavior, not a literal Git repository inside Firebase.

## 10 Global Versions

A Global Version is an owner-triggered immutable baseline of the current vault at a chosen moment. This is distinct from automatic Prompt-local history.

The current snapshot model can preserve profile, activity days/stats, achievement unlocks, Endeavors, Tasks, Prompts, Prompt Versions, attachments, relations, Finder feedback, Mindsets, Preferences, legacy Local Commit records and Decisions.

The Global Versions page supports release, browse, JSON download, metadata edit, archive/delete and comparison. Comparison reports Prompt count delta, local-version count delta and changed Prompt-state count/list. Snapshot breadth has evolved as the product gained new collections: files and relationships were added to snapshot coverage on 8 August; Finder feedback joined on 18 August.

## 11 Prompt Attachments

Prompt file support was introduced on 8 August 2026 in commit `3920b8d`, followed by a corrective commit `4e0310d`. Files are currently stored inline as Base64 in Firebase Realtime Database.

The attachment validator rejects empty files, files larger than 2 MiB, selections that would exceed 20 files on a Prompt, and selections that would exceed 10 MiB total. Download reconstructs a browser Blob. Presentation classifies images, PDFs, text/code-like files, archives and generic files.

Attachments are incorporated into Global Version snapshots and Prompt duplication behavior.

# 12 Prompt Lineage and Relationship Map

Prompt relationships were introduced on 8 August 2026 in commit `c09415a`; plotting was corrected shortly afterward by `37dc892`.

## 12.1 Validation invariants

Both endpoints must be active. A Prompt cannot inspire itself. Duplicate directed edges are forbidden. Before inserting an edge, the system checks whether a path already exists from the proposed child back to the proposed parent; if so, the edge would create a cycle and is rejected.

## 12.2 Visualization

The graph groups related Prompts by Endeavor and calculates ranks/positions for Endeavor groups and Prompt nodes. It supports cross-Endeavor edges, orthogonal routing and downloadable SVG/PNG output. A readable ledger provides a text-oriented view with search/filtering and navigation back to Prompt detail.

# 13 Mindsets, Mindset Construction and Preferences

Mindsets are scoped records with global, Endeavor, Task or Prompt scope. Preferences are scoped globally, by Endeavor or by Task.

## 13.1 Mindset Construction

The deterministic Mindset Builder is verified present by commit `417b29d` on 7 August 2026. The user searches/selects source Prompts, constructs an assembled persona/methodology draft, edits it and saves it as a normal Mindset. The builder explicitly states that no AI model is used. A constructed Mindset records source Prompt IDs and `constructionMethod = prompt-selection`.

This distinction matters: derived text in EurekaVault is not automatically AI-generated. The Mindset Builder is deterministic while Finder, Repurposer and Mixer are Gemini-backed.

# 14 Decision Log

Decisions store product/architecture questions and Open/Finalized state. Current provider normalization upgrades several early decision seeds: the old unlimited-folders decision becomes the direct hierarchy decision; the old manual Prompt-version policy becomes automatic Prompt history; and the old two-commit-level language becomes Prompt-local plus vault-global versioning.

Historical documentation saying no AI calls exist is superseded. Collaboration and markup format remain open/gated. Prompt Rating and Prompt Blocks were not found in the current code and therefore are not documented as implemented.

# 15 Activity Tracking and Achievements

Activity tracking and achievements were introduced on 8 August 2026 in commit `6f45937`. Activity days are distinct local calendar days and are cumulative rather than streak-only.

### 15.1 Tracked action families

The current action type supports session open, generic record create/update/archive/restore/delete, Prompt commit, Global Version release, Decision change, attachment add/remove/download, relationship add/update/remove/map download, Finder search, Finder feedback, Repurposer generation and Mixer generation.

### 15.2 Current achievements

**1st Prompt Commit** First meaningful Prompt history commit.
**1st Global Commit** First Global Version release.
**1st Mindset** First Mindset creation.
**1st Endeavour** First Endeavor creation.
**1 Week of Activity** Seven distinct active calendar days, not necessarily consecutive.
**30 Days of Activity** Thirty distinct active calendar days.
**Builder** At least one active Prompt longer than 500 characters.
**Fussy Builder** At least 75 percent of active Prompts longer than 500 characters.
**Skeptical** First Decision status/resolution revision.

Unlocks are persisted with an unlock timestamp and progress-at-unlock so an earned achievement remains earned even if live vault state changes later.

## 16 Conventional Search and Navigation

The deterministic Search page is separate from AI Finder. It searches active Prompts against title, description, purpose, current content and saved Prompt-version history. An optional Endeavor filter narrows results. The shared word matcher requires all query words to appear somewhere in the Prompt/history corpus.

The application shell also supplies broader navigation, responsive layout, theme controls and a command palette. These interaction surfaces were established through the GUI-overhaul sequence on 7 August 2026.

## 17 AI System Overview

The AI layer uses three Netlify Functions so the Gemini API key stays outside the browser. Client calls carry a Firebase ID token and UID. Each function verifies the token and requires the verified Firebase local ID to match the requested UID before calling Gemini.

Server-side environment variables include `GEMINI_API_KEY`, optional model overrides and an optional `FIREBASE_WEB_API_KEY`. The Gemini key must never be prefixed with `VITE_`.

## 18 Semantic Prompt Finder

The initial Finder was introduced on 8 August 2026 in commit `2038bdb` (*prompt retrieval*). It is semantic retrieval over existing Prompts, not Prompt execution.

### 18.1 Corpus construction

The client includes at most 200 active Prompts. Each Prompt contributes ID, title, description, purpose, bounded content, Task, Endeavor and direct relationship peers. A Prompt body is compacted to at most 24,000 characters, preserving the beginning and ending when truncation is required.

The current query limit is 2,000 characters.

### 18.2 Data explicitly excluded

Finder does not send attachments, version history, profile, Mindsets, Preferences or Decisions.

### 18.3 Prompt injection defense

The function system instruction classifies the stored corpus as untrusted data and forbids following instructions inside it. Returned IDs must belong to the supplied corpus. The server validates IDs, deduplicates them, bounds reasons, normalizes scores and returns at most five matches.

The score is explicitly approximate AI relevance rather than vector similarity. The Gemini interaction uses `store: false` in the inspected function.

## 19 Finder Feedback Learning

On 18 August 2026 commit `6818b80` added the latest implemented AI-side feature: explicit retrieval feedback that becomes few-shot guidance for future Finder searches.

### 19.1 User interaction

The user can press *This is it* on a returned match or choose any active Prompt and save it as the correct result. The search query, confirmed Prompt, historical match list, model/corpus metadata and learning-example count are persisted.

### 19.2 Client learning-example builder

Feedback is sorted newest-first. The selected Prompt must still exist and be active. Each historical query is capped at 800 characters. Duplicate query/Prompt pairs are removed. At most 24 examples are retained and the combined example text budget is bounded around 12,000 characters.

### 19.3 Server revalidation

The retrieval function sanitizes examples again and accepts a confirmed Prompt only if its ID is present in the current request's authoritative Prompt corpus. This prevents stale historical feedback from forcing removed/archived IDs.

### 19.4 Model instruction semantics

Gemini is told that confirmed examples are user-specific retrieval preference evidence: they help interpret how this owner describes intent and which distinctions matter. They are not allowed to override the current query or corpus, and historical example text is treated as data, not instructions.

This creates a bounded adaptive retrieval loop without fine-tuning a model or building a separate vector database.

## 20 Prompt Repurposer

Repurposer was introduced on 8 August 2026 in commit `fe018b7`. The conceptual transformation is Original Prompt Y + Objective X $\rightarrow$ Candidate Z.

The source includes title, description, purpose, content, Task and Endeavor. The user supplies the new objective. The transformation rule asks Gemini to preserve structure, constraints, formatting, specificity and detail as much as possible and change only what is necessary for the new objective.

The returned title, description, purpose and Prompt body are editable. Generation never modifies the source Prompt. Saving creates a new Prompt under a selected Endeavor/Task and therefore creates independent Version 1 history. No lineage link is auto-created.

## 21 Prompt Mixer

The initial Mixer was introduced on 8 August 2026 in commit `40fc00e`. Later the same day, commit `6923740` generalized Mixer sources so a source no longer had to be an existing vault Prompt.

At least two non-empty source windows are required. Each window can load an active vault Prompt or contain arbitrary pasted/typed Prompt text. An optional direction provides synthesis emphasis.

The mixing contract says to use every non-empty source, preserve distinct requirements/detail, consolidate real duplication and resolve conflicts into one coherent standalone Prompt. The UI explicitly excludes relationships, versions, attachments, Mindsets, Preferences, activity and achievements from this AI payload.

The generated title, description, purpose and full Prompt content are editable. The result can be discarded, copied, saved as a new Prompt or saved as the next version of an existing Prompt. The existing-Prompt path rejects an identical output so a meaningless version is not created.

## 22 AI Data Minimization and Failure Isolation

The AI tools do not send the entire user subtree. Each builds a purpose-specific payload. Finder has the most context because it must retrieve from active Prompts and can include direct relationship evidence plus bounded confirmed examples. Repurposer sends only one selected source plus objective. Mixer sends only its source windows and optional direction.

If Gemini is unavailable, quota-limited or returns malformed output, the feature reports an AI-specific error while the normal Firebase-backed vault remains available. This isolates AI availability from core CRUD/versioning access.

## 23 Settings, Diagnostics and Data Portability

Settings supports workspace-name editing, displays the owner email, checks Realtime Database connection state, Firebase session, email-verification state and workspace profile presence, and can resend verification when necessary.

The owner can export a local JSON snapshot of the current workspace. The page also displays the required EurekaVault Firebase rule as an operational reminder. Import/migration adapters are not implemented as a corresponding general restore workflow in the current inspected code.

## 24 Deployment and Configuration

Netlify runs `npm run build`, publishes `dist`, uses Node 22.12.0 and redirects all browser routes to `/index.html` for SPA routing.

Client Firebase settings may be supplied using `VITE_FIREBASE_*` environment variables. The repository also contains a fallback Firebase configuration. Gemini server variables are configured in Netlify and include the required `GEMINI_API_KEY` plus optional Finder, Repurposer and Mixer model overrides.

Netlify Functions are not a general backend. They are currently the server-side adapter needed for authenticated Gemini access and secret isolation. If a conventional backend is introduced, moving these Gemini calls and secret handling to that backend would simplify the architecture.

## 25 Testing and Validation

The project uses Vitest. Current test files include AI tests for mix, Prompt index, repurpose and retrieval plus library tests for achievements, attachments, diff, relationships and utilities.

The 18 August feedback change extends Prompt-index tests to verify recency, invalid-target filtering, current-title use and historical query bounds.

Standard validation commands are `npm run lint`, `npm run test` and `npm run build`. No dedicated Playwright/Cypress end-to-end suite, Firebase emulator integration suite or live Gemini integration suite was identified in the inspected repository tree, so external-service/browser integration still requires manual or deployment-level verification.

## 26 Implementation Chronology

The following table records the implementation chronology most relevant to current product functionality. It does not assign product release numbers that are not explicitly established in the repository.

| Date | Commit | Message | Significance |
|---|---|---|---|
| 2026-08-06 | `5eeec5f` | Initial commit | Repository begins with a minimal README. |
| 2026-08-06 | `c80d143` | Converting the project to react | Establishes the React/TypeScript/Vite application foundation. |
| 2026-08-06 | `c35286a` | Fixing Files | Early project/file correction during React scaffold setup. |
| 2026-08-06 | `18ab918` | adding CRUD features | Adds core CRUD/lifecycle capability and the practical data-management baseline. |
| 2026-08-07 | `417b29d` | Adding copy, versioning | Adds Prompt copy/versioning; Mindset Construction is verified present in this commit snapshot. |
| 2026-08-07 | `fca6117` | GUI overhaul | Begins major application UI reorganization. |
| 2026-08-07 | `8fcb57b` | changing-gui | Continues UI/navigation evolution. |
| 2026-08-07 | `81e662f` | overhaul GUI | Completes another major UI restructuring pass. |
| 2026-08-07 | `2a5c345` | fix error | Corrective commit after the GUI changes. |
| 2026-08-08 | `2b0f270` | adding back copy functionality | Restores Prompt-copy behavior following UI refactors. |
| 2026-08-08 | `1ac483e` | Github Line Diff | Adds Git-style line-diff review for Prompt versions. |
| 2026-08-08 | `6f45937` | Achievements feature | Adds activity tracking and achievements. |
| 2026-08-08 | `3920b8d` | Adding Files Functionality | Adds Prompt attachments and integrates them into vault lifecycle/snapshots. |
| 2026-08-08 | `4e0310d` | Fixing an error | Immediate correction after file functionality. |
| 2026-08-08 | `c09415a` | Adding Relationships between prompts | Adds Prompt lineage relationships, relationship UI/map integration and snapshot support. |

| Date | Commit | Message | Significance |
|---|---|---|---|
| 2026-08-08 | `37dc892` | fixing relationship plotting | Corrects relationship visualization/layout. |
| 2026-08-08 | `2038bdb` | prompt retrieval | Introduces Gemini-backed Semantic Prompt Finder via Netlify Function. |
| 2026-08-08 | `fe018b7` | Improving AI prompts | Introduces Prompt Repurposer and related Gemini server-side path. |
| 2026-08-08 | `40fc00e` | Prompt Mixer Feature | Introduces multi-Prompt Gemini synthesis. |
| 2026-08-08 | `6923740` | Prompt Mixer generic instead of forcing existing ones | Generalizes Mixer sources so arbitrary pasted/typed Prompts are first-class inputs. |
| 2026-08-18 | `6818b80` | feat: implementing feedback for AI search | Adds user-confirmed Finder feedback storage and bounded few-shot learning examples for later retrievals. |

## 27 Feature Implementation Ledger

This table is intentionally explicit about attribution. *Exact commit* means the repository history has a clear introducing commit for that capability. *Present by* means a commit snapshot proves the feature existed by that date even though the commit bundled multiple changes.

| Capability | Date | Commit | Attribution |
|---|---|---|---|
| React/TypeScript application foundation | 2026-08-06 | `c80d143` | Exact commit |
| Core CRUD and lifecycle baseline | 2026-08-06 | `18ab918` | Exact commit |
| Prompt copy + Prompt versioning | 2026-08-07 | `417b29d` | Exact commit |
| Mindset Construction | 2026-08-07 | `417b29d` | Verified present in commit snapshot |
| Major GUI/navigation overhaul | 2026-08-07 | `fca6117` / `8fcb57b` / `81e662f` | Commit sequence |
| Git-style Prompt line diff | 2026-08-08 | `1ac483e` | Exact commit |
| Achievements + activity tracking | 2026-08-08 | `6f45937` | Exact commit |
| Prompt file attachments | 2026-08-08 | `3920b8d` | Exact commit |
| Prompt relationship lineage | 2026-08-08 | `c09415a` | Exact commit |
| Relationship plotting correction | 2026-08-08 | `37dc892` | Exact fix commit |
| Semantic Prompt Finder | 2026-08-08 | `2038bdb` | Exact commit |
| Prompt Repurposer | 2026-08-08 | `fe018b7` | Exact commit |
| Prompt Mixer | 2026-08-08 | `40fc00e` | Exact commit |
| Mixer arbitrary pasted/custom sources | 2026-08-08 | `6923740` | Exact commit |
| Finder feedback-learning / few-shot retrieval adaptation | 2026-08-18 | `6818b80` | Exact commit |

## 28 Current Non-Implemented or Gated Areas

No current implementation was found for collaborative multi-user editing/synchronization, a finalized markup parser, Prompt Rating or Prompt Blocks. Collaboration and markup are explicitly historical open/gated areas. Rating and Prompt Blocks should remain outside implementation documentation until corresponding code exists.

The current architecture also does not provide a conventional backend service beyond Netlify Functions for AI.

## 29 Terminology Reference

**Prompt Version** Automatic historical snapshot of one Prompt.
**Global Version** Deliberate vault-wide baseline released by the owner.
**Local Commit / Global Commit** Legacy type/terminology still visible in portions of the data model; current UI emphasizes Prompt Versions and Global Versions.
**Inspired by** Child-facing direction of a Prompt relationship.
**Inspires** Parent-facing direction of the same relationship.
**Semantic Prompt Finder** Gemini-backed retrieval of existing Prompts.
**Finder feedback learning** Stored query-to-confirmed-Prompt examples used as bounded few-shot retrieval preference evidence.
**Repurposer** One-source controlled transformation into a new Prompt candidate.
**Mixer** Multi-source Prompt synthesis from vault or custom source windows.

## A Current Firebase User-Root Shape

```
intellectVault/users/{{uid}}/
  profile
  activityDays
  activityStats
  achievements
  endeavors
  tasks
  prompts
  promptVersions
  promptAttachments
  promptRelations
  promptFinderFeedback
  mindsets
  preferences
  localCommits
  globalCommits
  decisions
```

## B Current Attachment Limits

- 2 MiB maximum per file.
- 20 attachments maximum per Prompt.
- 10 MiB maximum aggregate attachment data per Prompt.

## C Current Finder Bounds

- 200 active Prompts maximum in the retrieval index.
- 24,000 characters maximum indexed content per Prompt (with head/tail compaction).
- 2,000 characters maximum current query.
- 24 historical confirmed examples maximum.
- 800 characters maximum per historical example query.
- Approximately 12,000 characters aggregate client-side learning-example budget.
- Five returned matches maximum.

## D Current Achievement IDs

```
first-prompt-commit
first-global-commit
first-mindset
first-endeavor
active-7-days
active-30-days
builder
fussy-builder
skeptical
```

## E Current AI Activity Actions

```
ai.prompt-finder.searched
ai.prompt-finder.feedback
ai.prompt-repurpose.generated
ai.prompt-mixer.generated
```

## F Primary Code Locations

- `src/App.tsx` - authenticated routes and provider composition.
- `src/context/AuthContext.tsx` - Firebase authentication and workspace initialization.
- `src/context/VaultContext.tsx` - vault subscription and mutation orchestration.
- `src/types/domain.ts` - persisted domain contracts.
- `src/pages/PromptWorkspacePage.tsx` - Prompt editor/history/files/relationships.
- `src/pages/CommitsPage.tsx` - Global Versions UI.
- `src/pages/RelationshipMapPage.tsx` - Prompt lineage map/ledger.
- `src/pages/MindsetConstructionPage.tsx` - deterministic Mindset Builder.
- `src/pages/AiPromptFinderPage.tsx` - Finder and feedback UI.
- `src/pages/AiPromptRepurposePage.tsx` - Repurposer UI.
- `src/pages/AiPromptMixerPage.tsx` - Mixer UI.
- `src/ai/promptIndex.ts` - AI source/index/example construction and bounds.
- `src/lib/attachments.ts` - attachment validation/encoding/download.
- `src/lib/relationships.ts` - relation validation, graph layout and export.
- `src/lib/achievements.ts` - achievement definitions/progress.
- `src/lib/diff.ts` - Prompt text diff engine.
- `netlify/functions/prompt-retrieval.mjs` - Finder Gemini boundary.
- `netlify/functions/prompt-repurpose.mjs` - Repurposer Gemini boundary.
- `netlify/functions/prompt-mix.mjs` - Mixer Gemini boundary.